

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información

Curso: Sistemas Operativos II



Módulos cargables del kernel y sistema de archivos /proc

Nombre: Juan José Suar Panjoj

Catedrático: Ing. René Ornelis

Fecha: 30 de abril, 2026

1. Introducción

Los módulos cargables del kernel (LKM) permiten extender las funcionalidades del sistema operativo Linux sin necesidad de reiniciar el sistema, estos módulos son fundamentales para la administración del sistema, ya que permiten agregar soporte para hardware, sistemas de archivos y otras funcionalidades de manera dinámica.

2. Objetivos

General: Comprender e implementar módulos cargables del kernel en Linux.

Específicos:

- Implementar un módulo tipo Hello World
- Crear un archivo en /proc
- Analizar comandos del sistema para manejo de módulos

3. Marco Teórico

Estructura básica de un módulo kernel en Linux

Un módulo cargable del kernel, conocido como **LKM**, es un componente de software que permite agregar funcionalidades al kernel de Linux sin necesidad de recompilarlo por completo ni reiniciar el sistema, estos módulos se compilan normalmente como archivos con extensión **.ko** y pueden cargarse o descargarse dinámicamente mediante comandos como `insmod`, `modprobe` y `rmmod`.

La estructura básica de un módulo del kernel se divide principalmente en dos partes: el archivo fuente **.c** y, en proyectos más organizados, uno o varios archivos de cabecera **.h**.

Archivo .c

El archivo **.c** contiene la implementación principal del módulo. En este archivo se escriben las funciones que ejecutará el kernel al momento de cargar y descargar el módulo. También se incluyen las bibliotecas necesarias del kernel y las macros que describen el módulo.

Un módulo básico en C suele incluir los siguientes elementos:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
```

`linux/module.h` permite trabajar con módulos del kernel y usar macros como `MODULE_LICENSE` y `MODULE_AUTHOR`.

`linux/kernel.h` incluye funciones y definiciones propias del kernel, como `printk`, que sirve para imprimir mensajes en el registro del sistema.

linux/init.h permite utilizar las macros __init y __exit, usadas para marcar funciones de inicialización y salida del módulo.

Después de las librerías, normalmente se definen las funciones principales:

```
static int __init hello_init(void)
{
    printk(KERN_INFO "Hello World: modulo cargado\n");
    return 0;
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "Hello World: modulo removido\n");
}
```

La función hello_init se ejecuta cuando el módulo es cargado en el kernel, su objetivo es inicializar recursos, crear estructuras, registrar dispositivos o, en este caso, mostrar un mensaje indicando que el módulo fue cargado correctamente.

La función hello_exit se ejecuta cuando el módulo es removido del kernel, su función es liberar recursos, eliminar archivos creados, des registrar dispositivos o mostrar un mensaje de salida.

Luego se conectan estas funciones con el kernel mediante:

```
module_init(hello_init);
module_exit(hello_exit);
```

Finalmente se agregan las macros de información:

```
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Juan_Suar");
MODULE_DESCRIPTION("Modulo Hello World del kernel");
```

Estas macros no forman parte de la lógica principal del programa, pero son importantes porque documentan el módulo y permiten que el kernel sepa detalles como la licencia, el autor y la descripción.

Archivo .h

El archivo .h, o archivo de cabecera, se utiliza para declarar funciones, constantes, estructuras o macros que serán usadas por uno o varios archivos.c; en módulos pequeños, como un "Hello World", no siempre es obligatorio utilizar un .h, pero en proyectos más grandes ayuda a mantener el código ordenado y reutilizable.

Un archivo .h puede contener definiciones como:

```
#ifndef HELLO_PROC_H
#define HELLO_PROC_H

#define PROC_NAME "hello_kernel"

int crear_archivo_proc(void);
void eliminar_archivo_proc(void);

#endif
```

Las líneas:

```
#ifndef HELLO_PROC_H
#define HELLO_PROC_H
#endif
```

son guardas de inclusión, sirven para evitar que el mismo archivo de cabecera sea incluido más de una vez durante la compilación, lo cual podría causar errores.

La constante:

```
#define PROC_NAME "hello_kernel"
```

define el nombre del archivo que será creado dentro de /proc.

Las declaraciones:

```
int crear_archivo_proc(void);
void eliminar_archivo_proc(void);
```

indican que existen funciones con esos nombres, aunque su implementación real puede estar en un archivo.c.

En resumen, el archivo .c contiene la lógica del módulo, mientras que el archivo .h contiene declaraciones y definiciones que ayudan a organizar el código.

Macros esenciales de un módulo kernel

MODULE_LICENSE

La macro MODULE_LICENSE especifica la licencia bajo la cual se distribuye el módulo, esto es importante porque el kernel de Linux identifica si el módulo utiliza una licencia compatible con GPL.

Ejemplo:

```
MODULE_LICENSE("GPL");
```

Usar "GPL" indica que el módulo utiliza una licencia compatible con el kernel de Linux, si no se coloca esta macro, el kernel puede mostrar una advertencia indicando que el módulo no tiene licencia definida, además, algunos símbolos internos del kernel solo están disponibles para módulos con licencia compatible GPL.

Esta macro no ejecuta una acción visible para el usuario, pero sí es importante para la integración correcta del módulo con el kernel.

MODULE_AUTHOR

La macro MODULE_AUTHOR permite indicar el autor del módulo, su función principal es documentar quién desarrolló el código.

Ejemplo:

```
MODULE_AUTHOR("Juan");
```

Esta información puede consultarse posteriormente con herramientas como:

```
modinfo hello_proc.ko
```

Aunque no afecta directamente el funcionamiento del módulo, es una buena práctica porque ayuda a identificar la autoría del código.

module_init()

La macro module_init() registra la función que se ejecutará cuando el módulo sea cargado al kernel.

Ejemplo:

```
module_init(hello_init);
```

En este caso, cuando se ejecuta:

```
sudo insmod hello_proc.ko
```

el kernel llama automáticamente a la función hello_init.

La función registrada con module_init() normalmente devuelve un entero, si retorna 0, significa que el módulo se cargó correctamente. Si retorna un valor negativo, significa que ocurrió un error durante la carga.

Ejemplo:

```
static int __init hello_init(void)
{
    printk(KERN_INFO "Hello World: modulo cargado\n");
}
```

```
    return 0;
}
```

Esta función puede realizar tareas como crear archivos en /proc, reservar memoria, registrar controladores, inicializar variables o preparar estructuras del kernel.

module_exit()

La macro `module_exit()` registra la función que se ejecutará cuando el módulo sea descargado del kernel.

Ejemplo:

```
module_exit(hello_exit);
```

Cuando se ejecuta:

```
sudo rmmod hello_proc
```

el kernel llama automáticamente a la función `hello_exit`.

Esta función no devuelve ningún valor porque su propósito principal es limpiar y liberar los recursos utilizados por el módulo.

Ejemplo:

```
static void __exit hello_exit(void)
{
    printk(KERN_INFO "Hello World: modulo removido\n");
}
```

En un módulo que crea un archivo en /proc, la función de salida debe eliminar ese archivo para evitar que queden recursos activos después de descargar el módulo.

4. Desarrollo Experimental

Se desarrolló un módulo en lenguaje C que imprime mensajes en el kernel y crea un archivo en /proc llamado `hello_kernel` este archivo muestra un mensaje estático al ser leído.

5. Comandos Utilizados

make: Compila el módulo

```
juan@juan-VirtualBox:~/tarea1_modulo$ make
make -C /lib/modules/6.17.0-22-generic/build M=/home/juan/tarea1_modulo modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.17.0-22-generic'
make[2]: se entra en el directorio '/home/juan/tarea1_modulo'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
make[2]: se sale del directorio '/home/juan/tarea1_modulo'
make[1]: se sale del directorio '/usr/src/linux-headers-6.17.0-22-generic'
```

insmod: Inserta el módulo en el kernel

```
juan@juan-VirtualBox:~/tarea1_modulo$ sudo insmod hello_proc.ko
```

lsmod: Lista módulos cargados

```
juan@juan-VirtualBox:~/tarea1_modulo$ lsmod | grep hello_proc
hello_proc          12288  0
```

ls: Verificación de la creación del archivo en el sistema /proc.

```
juan@juan-VirtualBox:~/tarea1_modulo$ ls /proc | grep hello_kernel
hello_kernel
```

cat: Lectura del archivo en /proc mostrando el mensaje "Hello World"

```
juan@juan-VirtualBox:~/tarea1_modulo$ cat /proc/hello_kernel
Hello Worldjuan@juan-VirtualBox:~/tarea1_modulo$ sudo dmesg | tail
```

Sudo: Visualización de mensajes del kernel tras la ejecución del módulo.

```
juan@juan-VirtualBox:~/tarea1_modulo$ cat /proc/hello_kernel
Hello Worldjuan@juan-VirtualBox:~/tarea1_modulo$ sudo dmesg | tail
[ 668.113185] audit: type=1400 audit(1777504895.024:174): apparmor="STATUS" operation="profile_replace" profile="unconfined" name="snap.firmware-updater.firmware-updater" pid=4789 comm="apparmor_parser"
[ 669.124105] audit: type=1400 audit(1777504896.038:175): apparmor="STATUS" operation="profile_replace" profile="unconfined" name="snap.firmware-updater.firmware-updater.app" pid=4790 comm="apparmor_parser"
[ 669.459295] audit: type=1400 audit(1777504896.383:176): apparmor="STATUS" operation="profile_replace" profile="unconfined" name="snap.firmware-updater.hook.configure" pid=4791 comm="apparmor_parser"
[ 669.668263] audit: type=1400 audit(1777504896.506:177): apparmor="STATUS" operation="profile_replace" info="same as current profile, skipping" profile="unconfined" name="snap.mesa-2404.component-monitor" pid=4792 comm="apparmor_parser"
[ 669.876339] audit: type=1400 audit(1777504896.801:178): apparmor="STATUS" operation="profile_replace" info="same as current profile, skipping" profile="unconfined" name="snap.mesa-2404.hook.connect-plug-kernel-gpu-2404" pid=4793 comm="apparmor_parser"
[ 670.054529] audit: type=1400 audit(1777504896.979:179): apparmor="STATUS" operation="profile_replace" info="same as current profile, skipping" profile="unconfined" name="snap.mesa-2404.hook.disconnect-plug-kernel-gpu-2404" pid=4794 comm="apparmor_parser"
[ 670.264216] audit: type=1400 audit(1777504897.171:180): apparmor="STATUS" operation="profile_replace" info="same as current profile, skipping" profile="unconfined" name="snap.mesa-2404.hook.install" pid=4795 comm="apparmor_parser"
[ 670.429233] audit: type=1400 audit(1777504897.349:181): apparmor="STATUS" operation="profile_replace" info="same as current profile, skipping" profile="unconfined" name="snap.mesa-2404.hook.post-refresh" pid=4796 comm="apparmor_parser"
[ 1230.272268] HelloProc: modulo removido correctamente
[ 1238.650582] Hello World: modulo cargado
```

Rmmod: Eliminación del módulo del kernel utilizando rmmod.

```
juan@juan-VirtualBox:~/tarea1_modulo$ sudo rmmod hello_proc
```

Verificación final: Verificación de que el archivo en /proc ha sido eliminado correctamente.

```
juan@juan-VirtualBox:~/tarea1_modulo$ ls /proc | grep hello_kernel
juan@juan-VirtualBox:~/tarea1_modulo$
```

6. Resultados

El módulo fue cargado exitosamente y se verificó su funcionamiento mediante el comando `lsmod`. Además, el archivo en /proc mostró correctamente el mensaje "Hello World".

7. Conclusiones

Se logró comprender la estructura y funcionamiento de los módulos del kernel en Linux, así como la utilidad del sistema /proc para interactuar con el kernel.